SQL 실습예제 3장 연습문제2 0402

문제 03 다음과 같은 심화 질문에 대한 SQL문을 작성하시오.

(1)'박지성'이 구매한 도서의 출판사와 같은 출판사의 도서를 구매한 고객의 이름을 구하시오.

1단계 – 박지성이 구매한 도서의 출판사를 찾는다
2단계 – 그 출판사의 도서를 찾는다
3단계 – 그 도서를 구매한 고객의 이름을 찾는다

(2)두 개 이상의 서로 다른 출판사의 도서를 구매한 고객의 이름을 구하시오

(3)전체 고객의 30% 이상이 구매한 도서를 검색하시오

문제 04
마당서점 데이터베이스에서 주문에 관한 사항은 Orders 테이블에 저장되어 있다. Order 테이블을 사용하여 도서번호 (bookid) 가 1번인 책은 주문하였으나 2번과 3번 책은 주문하지 않은 깩의 아이디(custid)를 찾는 SQL문을 작성하시오

(다른 답)

문제 5. 다음 질의에 대해 DDL문과 DML문을 작성하시오.

(1)새로운 도서('스포츠세계', '대한미디어', 10000원)을 Book테이블에 삽입하시오. 삽입이 안될 경우 필요한 데이터가 더 있는지 찾아보시오

```
INSERT INTO BOOK VALUES(11, '스포츠세계', '대한미디어', 10000);
```

(2)출판사 '삼성당'에서 출간한 도서를 삭제하시오
(3)출판사 '이상미디어'에서 출판한 도서를 삭제하시오. 삭제되지 않을 경우 그 원인을 설명하시오.
(4)출판사 '대한미디어'를 '대한출판사'로 이름을 변경하시오
(5)출판사에 대한 정보를 저장하는 테이블 Bookcompany(name, address, begin)을 생성하시오. name은 기본키(VARCHAR2(20)), address는 VARCHAR2(20), begin은 DATE타입으로 선언하여 생성하시오.
(6)Bookcompany테이블에 인터넷 주소를 저장하는 webaddress 속성을 VARCHAR2(30)으로 추가하시오
(7)Bookcompany테이블에 임의의 튜플 name=한빛아카데미, address=서울시 마포구 , begin=1993-01-01, webaddress는 http://hanbit.co.kr을 삽입하시오.

문제 7.릴레이션 R(A,B,C), S(C,D,E)가 주어졌을 때 다음 관계대수를 같은 의미를 갖는 SQL문으로 변환하시오

(1) $\sigma_{A=a2}(R)$

(2)$\pi_{A,B}(R)$

(3)$R \bowtie_{R.c=S.c} S$

문제 8

다음 릴레이션 R(A, B, C)와 S(C,D,E)가 주어졌을 때 다음 결과가 도출되도록 sql문을 작성하시오

R

A	B	C
a1	b1	c1
a2	b1	c1
a3	b1	c2
a4	b2	c4

S

C	D	E
c1	d2	e1
c1	d1	e2
c2	d3	e3
c5	d3	e3

(1)SELECT * FROM R INNER JOIN S ON (R.c=S.c);

(2)SELECT * FROM R LEFT OUTER JOIN S ON (R.c=S.c);

(3)SELECT * FROM R RIGHT OUTER JOIN S ON (R.c=S.c);

(4)SELECT * FROM R FULL OUTER JOIN S ON (R.c=S.c);

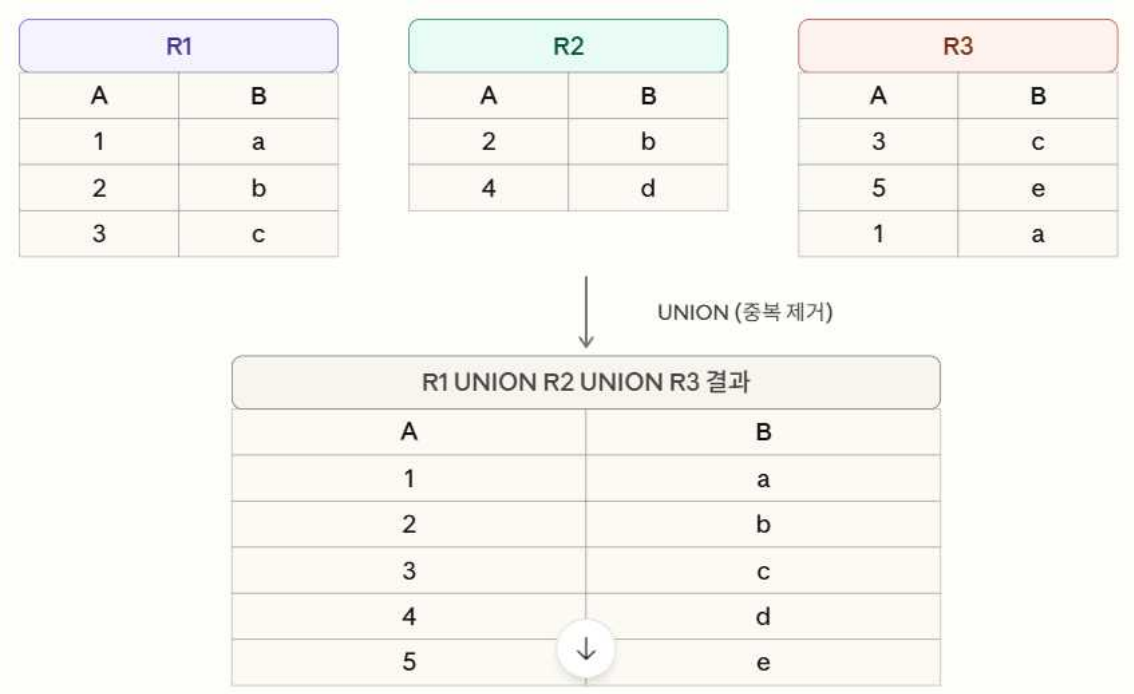
(5)SELECT * FROM R CROSS JOIN S ;

σ	셀렉션 (Selection)	σ_조건(R) — 조건을 만족하는 튜플만 선택
π	프로젝션 (Projection)	π_속성(R) — 특정 속성(열)만 추출
∪	유니온 (Union)	R ∪ S — 두 릴레이션의 합집합
∩	인터섹션 (Intersection)	R ∩ S — 두 릴레이션의 교집합
−	차집합 (Difference)	R − S — R에는 있고 S에는 없는 튜플
×	카티션 곱 (Cartesian product)	R × S — 모든 튜플의 조합
⋈	자연 조인 (Natural join)	R ⋈ S — 공통 속성 기준으로 자동 조인
⋈ θ	세타 조인 (Theta join)	R ⋈_θ S — 조건(θ) 기반 조인

R × S (크로스 조인) — 4 × 4 = 16행

R.A	R.B	R.C	S.C	S.D	S.E
a1	b1	c1	c1	d2	e1
a1	b1	c1	c1	d1	e2
a1	b1	c1	c2	d3	e3
a1	b1	c1	c5	d3	e3
a2	b1	c1	c1	d2	e1
a2	b1	c1	c1	d1	e2
a2	b1	c1	c2	d3	e3
a2	b1	c1	c5	d3	e3
a3	b1	c2	c1	d2	e1
a3	b1	c2	c1	d1	e2
a3	b1	c2	c2	d3	e3

문제 9. UNION 설명(UNION과 UNION ALL)




```
-- UNION: 중복 자동 제거
SELECT A, B FROM R1
UNION
SELECT A, B FROM R2
UNION
SELECT A, B FROM R3;

-- UNION ALL: 중복 포함 (더 빠름)
SELECT A, B FROM R1
UNION ALL
SELECT A, B FROM R2
UNION ALL
SELECT A, B FROM R3;
```

문제 10. EXISTS와 NOT EXISTS 예제

(1)주문한 적 있는 고객 조회

← 이전 1 / 4단계 다음 → custid=1 → Orders에 주문 2건 발견! → EXISTS = TRUE → 반환

Customer (외부 쿼리)

custid	name
1	김철수
2	이영희
3	박민준
4	정수현

Orders (서브쿼리)

custid	saleprice
1	12000
1	8000
2	15000
3	9000

결과 (주문한 적 있는 고객)

name	판정
김철수	EXISTS TRUE → 반환
이영희	?
박민준	?
정수현	?

← 이전 2 / 4단계 다음 → custid=2 → Orders에 주문 1건 발견! → EXISTS = TRUE → 반환

Customer (외부 쿼리)

custid	name
1	김철수
2	이영희
3	박민준
4	정수현

Orders (서브쿼리)

custid	saleprice
1	12000
1	8000
2	15000
3	9000

결과 (주문한 적 있는 고객)

name	판정
김철수	EXISTS TRUE → 반환
이영희	EXISTS TRUE → 반환
박민준	?
정수현	?

← 이전 3 / 4단계 다음 → custid=3 → Orders에 주문 1건 발견! → EXISTS = TRUE → 반환

Customer (외부 쿼리)

custid	name
1	김철수
2	이영희
3	박민준
4	정수현

Orders (서브쿼리)

custid	saleprice
1	12000
1	8000
2	15000
3	9000

결과 (주문한 적 있는 고객)

name	판정
김철수	EXISTS TRUE → 반환
이영희	EXISTS TRUE → 반환
박민준	EXISTS TRUE → 반환
정수현	?

← 이전 4 / 4단계 다음 → custid=4 → Orders에 주문 없음 → EXISTS = FALSE → 제외

Customer (외부 쿼리)

custid	name
1	김철수
2	이영희
3	박민준
4	정수현

Orders (서브쿼리)

custid	saleprice
1	12000
1	8000
2	15000
3	9000

결과 (주문한 적 있는 고객)

name	판정
김철수	EXISTS TRUE → 반환
이영희	EXISTS TRUE → 반환
박민준	EXISTS TRUE → 반환
정수현	EXISTS FALSE → 제외

** EXISTS와 NOT EXISTS는 항상 상관 서브쿼리 형태로 사용되며 외부 쿼리의 각 행마다 서브쿼리를 실행한다.

** IN으로 대체 가능한 경우도 있지만, EXISTS는 서브쿼리 결과가 크거나 NULL이 포함된 경우에 더 안전하고 빠르다.

문제 17(218page)

마당서점의 테이블 구성은 다음과 같다.
(마당서점 모델)
Book(bookid, bookname, publisher, price)

Customer(custid, name, address, phone)
Orders(orderid, custid, bookid, saleprice, orderdate)
마당서점은 고객 주소 혹은 전화번호 변경을 위해 Cust_addr테이블을 추가하여고객의 주소변경 사항을 저장하고자 한다. Cust_addt의 기본키는 (custid, addrid)이고, custid 속성은 Customer의 custid를 참조하는 외래키이다.
새로 변경된 마당서점 데이터베이스 구조를 보고 다음 질의에 대한 SQL문을 작성하시오. 날짜는 YYYY-MM-DD 형식으로 표기하시오

Book bookid, bookname, publisher, price

bookid PK	bookname	publisher	price
1	데이터베이스 개론	한빛미디어	28,000
2	자바의 정석	길벗	32,000
3	파이썬 머신러닝	위키북스	25,000
4	운영체제 원리	한빛미디어	30,000
5	알고리즘 문제 풀기	길벗	22,000
6	클린 코드	인사이트	33,000
7	SQL 첫걸음	한빛미디어	24,000

Customer custid, name, address, phone

custid PK	name	address	phone
1	박지수	서울 마포구 합정동	010-1234-5678
2	김도윤	경기 성남시 분당구	010-2345-6789
3	이서연	서울 강남구 역삼동	010-3456-7890
4	최민준	인천 부평구 부평동	010-4567-8901
5	정하은	서울 송파구 잠실동	010-5678-9012

(1)고객번호 1번의 주소 변경 내역을 모두 조회하시오
(2)고객번호 1번의 전화번호 변경 내역을 모두 조회하시오
(3)고객번호 1번의 가입 당시 전화번호를 조회하시오

Orders orderid, custid(FK→Customer), bookid(FK→Book), saleprice, orderdate

orderid PK	custid FK	bookid FK	saleprice	orderdate
1	1	1	28,000	2026-01-05
2	1	3	23,000	2026-01-08
3	2	2	32,000	2026-01-10
4	2	5	20,000	2026-01-12
5	3	4	30,000	2026-01-15
6	3	6	31,000	2026-01-15
7	4	7	22,000	2026-01-18
8	5	1	26,000	2026-01-20
9	5	2	32,000	2026-01-22
10	1	6	33,000	2026-01-25

Cust_addr NEW custid(PK·FK→Customer), addrid(PK), address, phone, changedate

custid PK FK	addrid PK	address	phone	changedate
1	1	서울 은평구 불광동	010-1234-5678	2024-03-10
1	2	서울 마포구 합정동	010-1234-5678	2025-07-20
2	1	경기 수원시 팔달구	010-2345-6789	2023-11-05
2	2	경기 성남시 분당구	010-9999-0000	2025-02-14
3	1	서울 강남구 역삼동	010-3456-7890	2025-05-01
4	1	부산 해운대구 우동	010-4567-8901	2024-08-22

(4)고객번호 1번의 “2025년 01월 01” 당시 전화번호를 보이시오. 주소 이력 중 changeday 속성값이 “2025년 01”보다 오래된 첫 번째 값을 찾는다.

addrid는 고객별 순번임, 고객별 순번

문제 18마당서점의 태이블 구성은 17번과 같다. 마당 서점을 Cart를 추가하여 장바구니 sms 고객이 주문 전에 도서를 미리 담아놓는 역할을 하며 Orders테이블과 비슷한 구조를 갖는다. Cart테입르의 기본 키는 cartid이고 , custid와 bookid는 각각 Customer. custid, 와 Book.bookid를 참조하는 외래키이다.

Cart(cartid, custid, biikid, cartdate)
새로 변경된 마당서점 데이터베이스 구조를 보고 다음 질의에 대한 SQL문을 작성하시오

(1)고객번호 1번의 cart에 저장된 도서 중 이미 주문한 도서를 구하시오
(2)고객번호 1번의 cart에 저장된 도서 중 아직 주문하지 않은 도서를 구사시오
(3)고객번호 1번의 cart에 저장된 도서들의 정가 합계를 구하시오.

문제 19. Emp, Dept테이블로 구성된 회사사원 데이터베이스를 만들고자 한다. 테이블을 생성하고 데이터를 입력하는 SQL질의를 작성하시오.
(1)부서(Department)dp 관한 Dept테이블은 depno(부서번호), dname(부서이름), loc(부서위치, location)으로 구성되어 있다. Dept테이블을 생성하는 SQL명령어는 다음과 같다. Dept 테이블을 생성하시오.

(2)사원(Employee)에 관한 Emp테이블은 empno(사원번호), ename(사원이름),job(업무), mgr(직속상사번호, manager) hiredate(고용날짜), sal(월급여, salary), comm(판매수당,commission), deptno(부서번호)로 구성되어 있다. Emp의 deptno는 Dept의 deptno를 참조하는 외래키로 지정한다. desc Emp;명령을 사용하면 테이블의 구조를 볼 수 있다. Emp테이블을 생성하는 SQL문을 작성하시오

DESC Dept; 부서 테이블

Field	Type	Null	Key	Default	Extra
deptno	NUMBER(2)	NO	PK	NULL	
dname	VARCHAR2(14)	YES		NULL	
loc	VARCHAR2(13)	YES		NULL	

DESC Emp; 사원 테이블

Field	Type	Null	Key	Default	Extra
empno	NUMBER(4)	NO	PK	NULL	
ename	VARCHAR2(10)	YES		NULL	
job	VARCHAR2(9)	YES		NULL	
mgr	NUMBER(4)	YES		NULL	자기참조 (empno)
hiredate	DATE	YES		NULL	
sal	NUMBER(7,2)	YES		NULL	
comm	NUMBER(7,2)	YES		NULL	없으면 NULL
deptno	NUMBER(2)	YES	FK	NULL	→ Dept.deptno

-- (1) Dept 테이블 생성 (먼저 생성해야 함)

-- (2) Emp 테이블 생성 (Dept가 먼저 존재해야 FK 설정 가능)

(3)부서에 관한 다음 네 개의 데이터를 삽입하시오.

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATION	BOSTON

(4사원에 관한 다음 네 개의 데이터를 삽입하시오

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	800	(null)	20
7499	ALLEN	SALESMAN	7698	1981-02-20	1600	300	30
7521	WARD	SALESMAN	7698	1981-02-22	1250	500	30
7566	JONES	MANAGER	7839	1981-04-02	2975	(null)	20

(5)사원 테이블에 다음 데이터를 삽입하려고 하니 오류가 발생하였다. 오류 메시지를 확인해 보고 원인을 찾아보시오

```
INSERT INTO Emp (empno,ename, job,mgr, hiredate,sal,comm,deptno ) values
(7654,'MARTIN', 'SALESMAN',7698, '1981-09-2800:00:00', 1250,1400,50);
```

[서브질의]

(6)각 사원의 사원이름(ename)과 소속 부서이름(dname), 부서위치(loc)를 함께 조회하시오

(7)부서가 배정된 사원의 사원번호(empno), 사원이름(ename), 부서이름(dname)을 조회하시오. 부서가 없는 사원은 제외한다.

(8)부서 위치(loc)가 'DALLAS'인 부서에 소속된 모든 사원의 이름(ename), 업무(job), 월급여(sal)를 조회하시오.

(9)부서 위치(loc)가 'DALLAS'인 부서에 소속된 사원의 이름(ename), 업무(job)를 조회하시오. 서브쿼리를 사용할 것.

(10)전체 사원의 평균 급여(AVG(sal))보다 급여가 높은 사원의 이름(ename)과 급여(sal)를 조회하시오.

(11)각 사원의 이름(ename)과 그 사원의 직속상사 이름을 함께 조회하시오. 상사가 없는 사원도 포함할 것.

(12)각 부서에서 급여가 가장 높은 사원의 이름(ename), 급여(sal), 부서이름(dname)을 조회하시오. 조인과 서브쿼리를 함께 사용할 것.

(13)부서 테이블의 구조를 변경하여 부서장의 이름을 저장하는 manager속성을 추가하고자 한다. ALTER 문을 사용하여 작성해 보시오. managername 속성이 만들어졌으면 UPDATE문을 이용하여 MANAGER의 이름을 입력하시오.